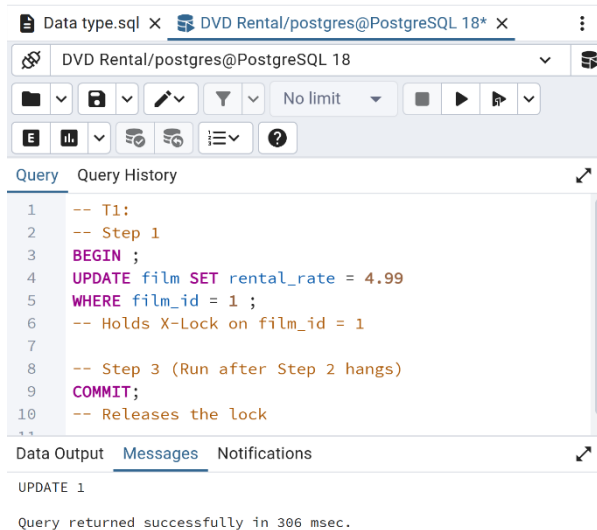


SECTION 6: CONCURRENCY CONTROL

Lab 1: Experiencing Blocking (Row-Level Lock).

Objective: Observe how Row-level Exclusive Locks (X-Locks) block other writers

- Run step 1:

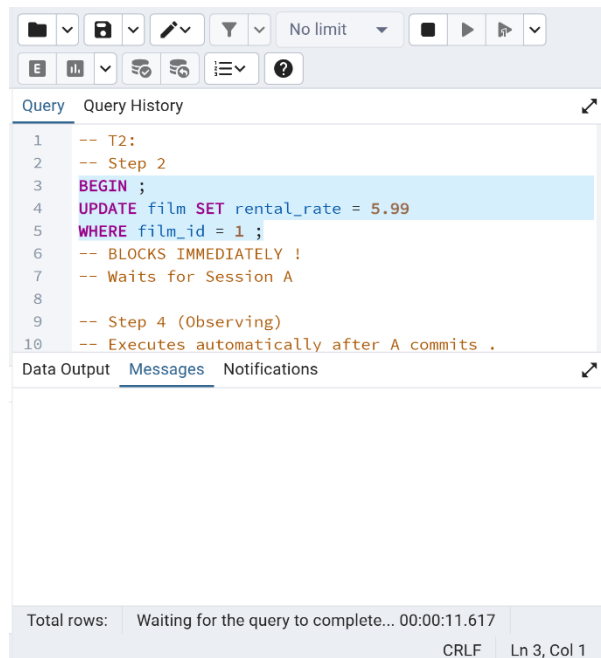


The screenshot shows a PostgreSQL client window with the title 'DVD Rental/postgres@PostgreSQL 18*'. The query editor contains the following SQL code:

```
1  -- T1:
2  -- Step 1
3  BEGIN ;
4  UPDATE film SET rental_rate = 4.99
5  WHERE film_id = 1 ;
6  -- Holds X-Lock on film_id = 1
7
8  -- Step 3 (Run after Step 2 hangs)
9  COMMIT;
10 -- Releases the lock
```

The 'Messages' tab is selected, showing the output: 'UPDATE 1' and 'Query returned successfully in 306 msec.'

- Run step 2:



The screenshot shows the same PostgreSQL client window. The query editor contains the following SQL code:

```
1  -- T2:
2  -- Step 2
3  BEGIN ;
4  UPDATE film SET rental_rate = 5.99
5  WHERE film_id = 1 ;
6  -- BLOCKS IMMEDIATELY !
7  -- Waits for Session A
8
9  -- Step 4 (Observing)
10 -- Executes automatically after A commits .
```

The 'Messages' tab is selected, but it is empty. At the bottom of the window, a status bar indicates: 'Total rows: Waiting for the query to complete... 00:00:11.617 CRLF Ln 3, Col 1'.

- Run step 3:

Query	Query History	Query	Query History
<pre> 1 -- T1: 2 -- Step 1 3 BEGIN ; 4 UPDATE film SET rental_rate = 4.99 5 WHERE film_id = 1 ; 6 -- Holds X-Lock on film_id = 1 7 8 -- Step 3 (Run after Step 2 hangs) 9 COMMIT; 10 -- Releases the lock </pre>	<pre> 1 -- T1: 2 -- Step 1 3 BEGIN ; 4 UPDATE film SET rental_rate = 4.99 5 WHERE film_id = 1 ; 6 -- Holds X-Lock on film_id = 1 7 8 -- Step 3 (Run after Step 2 hangs) 9 COMMIT; 10 -- Releases the lock </pre>	<pre> 1 -- T2: 2 -- Step 2 3 BEGIN ; 4 UPDATE film SET rental_rate = 5.99 5 WHERE film_id = 1 ; 6 -- BLOCKS IMMEDIATELY ! 7 -- Waits for Session A 8 9 -- Step 4 (Observing) 10 -- Executes automatically after A commits . 11 COMMIT; </pre>	<pre> 1 -- T2: 2 -- Step 2 3 BEGIN ; 4 UPDATE film SET rental_rate = 5.99 5 WHERE film_id = 1 ; 6 -- BLOCKS IMMEDIATELY ! 7 -- Waits for Session A 8 9 -- Step 4 (Observing) 10 -- Executes automatically after A commits . 11 COMMIT; </pre>
COMMIT Query returned successfully in 183 msec.	COMMIT Query returned successfully in 183 msec.	UPDATE 1 Query returned successfully in 2 min.	UPDATE 1 Query returned successfully in 2 min.
Total rows: Query complete 00:00:00.183 CRLF Ln 9, Col 1	Total rows: Query complete 00:00:00.183 CRLF Ln 9, Col 1	Total rows: Query complete 00:02:00.878 CRLF Ln 5, Col 20	Total rows: Query complete 00:02:00.878 CRLF Ln 5, Col 20

- Run step 4:

Query	Query History	Query	Query History
<pre> 1 -- T1: 2 -- Step 1 3 BEGIN ; 4 UPDATE film SET rental_rate = 4.99 5 WHERE film_id = 1 ; 6 -- Holds X-Lock on film_id = 1 7 8 -- Step 3 (Run after Step 2 hangs) 9 COMMIT; 10 -- Releases the lock </pre>	<pre> 1 -- T1: 2 -- Step 1 3 BEGIN ; 4 UPDATE film SET rental_rate = 4.99 5 WHERE film_id = 1 ; 6 -- Holds X-Lock on film_id = 1 7 8 -- Step 3 (Run after Step 2 hangs) 9 COMMIT; 10 -- Releases the lock </pre>	<pre> 1 -- Step 2 2 BEGIN ; 3 UPDATE film SET rental_rate = 5.99 4 WHERE film_id = 1 ; 5 -- BLOCKS IMMEDIATELY ! 6 -- Waits for Session A 7 8 9 -- Step 4 (Observing) 10 -- Executes automatically after A commits . 11 COMMIT; </pre>	<pre> 1 -- Step 2 2 BEGIN ; 3 UPDATE film SET rental_rate = 5.99 4 WHERE film_id = 1 ; 5 -- BLOCKS IMMEDIATELY ! 6 -- Waits for Session A 7 8 9 -- Step 4 (Observing) 10 -- Executes automatically after A commits . 11 COMMIT; </pre>
COMMIT Query returned successfully in 183 msec.	COMMIT Query returned successfully in 183 msec.	COMMIT Query returned successfully in 196 msec.	COMMIT Query returned successfully in 196 msec.
Total rows: Query complete 00:00:00.183 CRLF Ln 9, Col 1	Total rows: Query complete 00:00:00.183 CRLF Ln 9, Col 1	Total rows: Query complete 00:00:00.196 CRLF Ln 11, Col 1	Total rows: Query complete 00:00:00.196 CRLF Ln 11, Col 1

Lab 2: Replicating Non-repeatable Read.

Objective: See how data changes within the same transaction under the default Read Committed isolation level.

- Run step 1:

Query

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.99!
11 COMMIT;

```

Data Output

rental_rate	numeric (4,2)
1	4.99

Total rows: 1 Query complete 00:00:00.768 CRLF Ln 2, Col 1

- Run step 2:

Query

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.09!
11 COMMIT;

```

Data Output

rental_rate	numeric (4,2)
1	4.99

Total rows: 1 Query complete 00:00:00.768 CRLF Ln 10, Col 20

Query

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 0.99
4  WHERE film_id = 10;
5
6  COMMIT;
7  -- Modifies data and commits
8  -- while A is still active .
9

```

Data Output

COMMIT

Query returned successfully in 160 msec.

Total rows: Query complete 00:00:00.160 CRLF Ln 5, Col 1

- Run step 3:

The screenshot shows a PostgreSQL query editor with the following SQL code:

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.09!
11 COMMIT;

```

The "Data Output" tab is active, showing the result of the query:

	rental_rate numeric(4,2)
1	0.99

At the bottom, the status bar indicates: "Total rows: 1 | Query complete 00:00:00.323 | CRLF | Ln 8, Col 1".

Lab 3: Creating a Deadlock.

Objective: Force a circular wait and watch PostgreSQL kill the victim.

- Run step 1:

The screenshot shows a PostgreSQL query editor with the following SQL code:

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 100;
5
6  -- Step 3
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 200;
9  -- Blocks ( Waiting for B)

```

The "Messages" tab is active, showing the execution status:

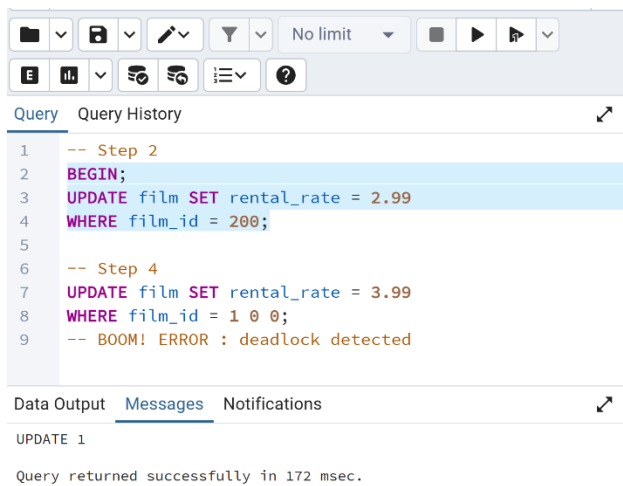
```

UPDATE 1
Query returned successfully in 172 msec.

```

At the bottom, the status bar indicates: "Total rows: | Query complete 00:00:00.172 | CRLF | Ln 2, Col 1".

- Run step 2:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and filters. Below the toolbar, the 'Query' tab is active, displaying a script with the following content:

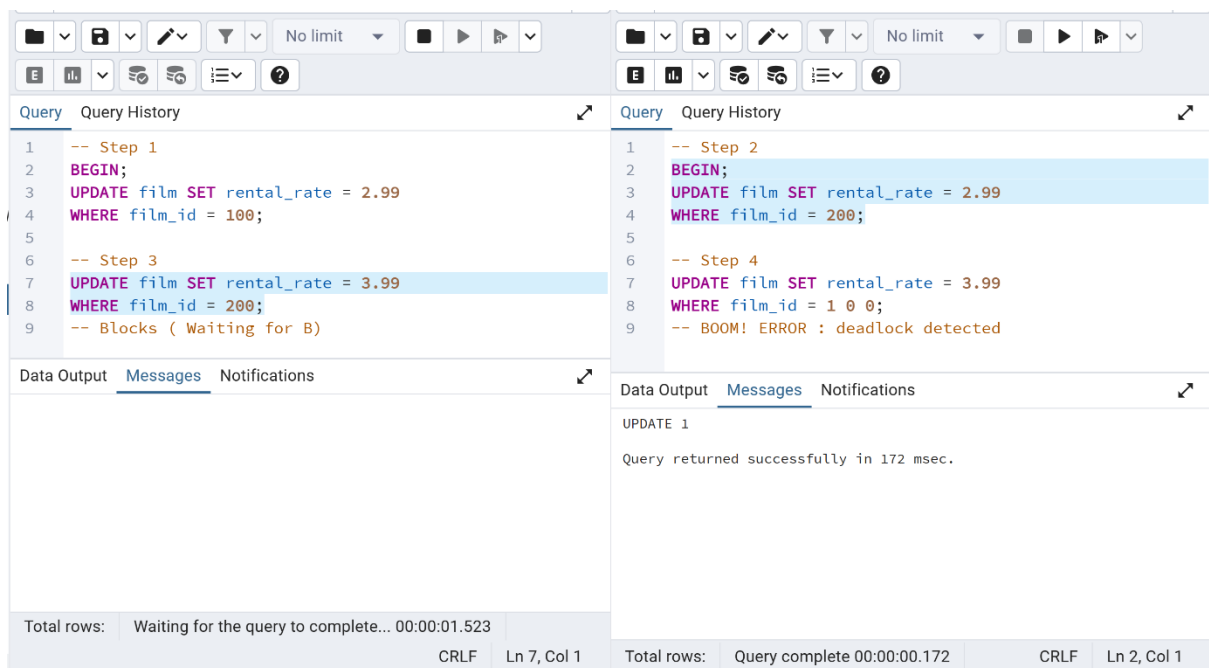
```
1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 200;
5
6  -- Step 4
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 100;
9  -- BOOM! ERROR : deadlock detected
```

The 'Messages' tab is also visible, showing the output of the query:

```
UPDATE 1
Query returned successfully in 172 msec.
```

At the bottom, the status bar indicates 'Total rows: Query complete 00:00:00.172 CRLF Ln 2, Col 1'.

- Run step 3:



The image displays two side-by-side screenshots of the SQL IDE, illustrating the execution of Step 3. The left screenshot shows the script with Step 3 highlighted:

```
1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 100;
5
6  -- Step 3
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 200;
9  -- Blocks ( Waiting for B)
```

The right screenshot shows the same script, but with Step 2 highlighted. The 'Messages' tab in the right IDE shows the output of the query:

```
UPDATE 1
Query returned successfully in 172 msec.
```

The status bar at the bottom of the right IDE indicates 'Total rows: Query complete 00:00:00.172 CRLF Ln 2, Col 1'.

- Run step 4:

🕒

DVD Rental/postgres@PostgreSQL 18

⌵

📁

📄

✎

🔍

No limit

📄

▶

🔍

⌵

E

📊

⌵

🔄

🔄

⋮

?

Query

Query History

↗

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 100;
5
6  -- Step 3
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 200;
9  -- Blocks ( Waiting for B)

```

Data Output

Messages

Notifications

↗

```
UPDATE 1
```

Query returned successfully in 28 secs 658 msec.

Total rows:

Query complete 00:00:28.751

CRLF

Ln 7, Col 1

🕒

DVD Rental/postgres@PostgreSQL 18

⌵

📁

📄

✎

🔍

No limit

📄

▶

🔍

⌵

E

📊

⌵

🔄

🔄

⋮

?

Query

Query History

↗

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 200;
5
6  -- Step 4
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 100;
9  -- BOOM! ERROR : deadlock detected

```

Data Output

Messages

Notifications

↗

```
ERROR:  deadlock detected
Process 14908 waits for ShareLock on transaction 987; blocked by
process 16596.
Process 16596 waits for ShareLock on transaction 988; blocked by
process 14908.

SQL state: 40P01
Detail: Process 14908 waits for ShareLock on transaction 987;
blocked by process 16596.
Process 16596 waits for ShareLock on transaction 988; blocked by
process 14908.
Hint: See server log for query details.
Context: while locking tuple (54.2) in relation "film"
```

Total rows:

Query complete 00:00:01.286

CRLF

Ln 7, Col 1

Lab 4: Preventing Race Conditions (FOR UPDATE).

Objective: Safely handle "Double Booking" when two customers try to rent the exact same DVD copy simultaneously.

- Run step 1:

The screenshot displays the pgAdmin interface. At the top, there's a toolbar with icons for file operations, query execution, and settings. Below this, the 'Query' tab is active, showing a SQL script. The script consists of several lines: a comment '-- Step 1', a 'BEGIN;' statement, a 'SELECT * FROM inventory WHERE inventory_id = 10 FOR UPDATE;' statement with a comment '-- Locks the row proactively', a comment '-- Step 3', an 'INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id) VALUES (NOW(), 10, 1, 1);' statement, and a 'COMMIT;' statement. The 'Data Output' tab is also visible, showing a table with 5 columns: 'inventory_id [PK] integer', 'film_id smallint', 'store_id smallint', and 'last_update timestamp without time zone'. The table contains one row of data: (10, 2, 2, 2006-02-15 10:09:17). The status bar at the bottom indicates 'Total rows: 1', 'Query complete 00:00:00.293', 'CRLF', and 'Ln 2, Col 1'.

```

-- Step 1
BEGIN ;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- Locks the row proactively

-- Step 3
INSERT INTO rental
(rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;

```

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

Total rows: 1 Query complete 00:00:00.293 CRLF Ln 2, Col 1

- Run step 2:

